

Set 1 - OpenMP, MPI, and Python

Software & Programming of HPC Systems

February 2026

Prof. Hadjidoukas

Achilleas Villiotis - 1084567

Winter 2025-2026

1 Parallel Numerical Optimization

1.1 Εισαγωγή & Μεθοδολογία

Στο πρώτο μέρος της άσκησης μελετήθηκε η παράλληλη υλοποίηση του αλγορίθμου **Multistart MDS** για την ελαχιστοποίηση της συνάρτησης **Rosenbrock**. Η σειριακή έκδοση αναφοράς (`multistart_mds_seq.c`) εκτελεί πολλαπλές, ανεξάρτητες επανεκκινήσεις (`starts`) της μεθόδου τοπικής αναζήτησης. Για την προσομοίωση ρεαλιστικού φόρτου εργασίας σε συστήματα υψηλής επίδοσης, έχει εισαχθεί τεχνητή καθυστέρηση `sleep(100us)` ανά υπολογισμό της συνάρτησης.

Οι παράλληλες στρατηγικές που υλοποιήθηκαν και αξιολογήθηκαν είναι οι εξής:

1. **OpenMP Parallel For:** Χρήση της οδηγίας `#pragma omp parallel for` για τον διαμοιρασμό των επαναλήψεων (`trials`) στα διαθέσιμα νήματα. Για τη συλλογή αποτελεσμάτων, χρησιμοποιήθηκε `custom reduction` ώστε να εντοπιστεί η βέλτιστη λύση (ελάχιστη τιμή συνάρτησης) και το αντίστοιχο διάνυσμα εισόδου μεταξύ των νημάτων.
2. **OpenMP Tasks:** Υλοποίηση με χρήση `tasks` (`#pragma omp task`) εντός ενός `single parallel region`. Κάθε δοκιμή (`trial`) ανατίθεται σε ένα `task`. Η προσέγγιση αυτή επιτρέπει δυναμική εξισορρόπηση φορτίου (`load balancing`), η οποία είναι ιδιαίτερα χρήσιμη όταν το κόστος κάθε δοκιμής δεν είναι σταθερό. Η ενημέρωση της βέλτιστης τιμής προστατεύεται με `#pragma omp critical`.
3. **MPI (Message Passing Interface):** Αρχιτεκτονική κατανεμημένης μνήμης όπου κάθε διεργασία (`rank`) αναλαμβάνει ένα υποσύνολο των δοκιμών. Η διανομή γίνεται κυκλικά (**rank-strided**) για ομοιόμορφη κατανομή φόρτου. Στο τέλος, χρησιμοποιείται `MPI_Reduce` με `custom struct/operation` για τη συλλογή του ολικού ελαχίστου και του συνολικού αριθμού υπολογισμών.

1.2 Αποτελέσματα & Σχολιασμός

Ο παρακάτω πίνακας συνοψίζει τους χρόνους εκτέλεσης και την επιτάχυνση (`Speedup`) για τις διάφορες υλοποιήσεις. Η επιτάχυνση υπολογίζεται ως $S = \frac{T_{\{seq\}}}{T_{\{par\}}}$, με $T_{\{seq\}} \approx 97.10$ s.

OpenMP For			OpenMP Task			MPI		
Cores	Time (s)	Speedup	Cores	Time (s)	Speedup	Cores	Time (s)	Speedup
1	97.108	1 ×	1	97.136	1 ×	1	97.092	1 ×
2	49.65	1.96 ×	2	48.92	1.98 ×	2	49.321	1.97 ×
4	24.681	3.93 ×	4	24.583	3.95 ×	4	24.661	3.94 ×
6	17.006	5.71 ×	6	16.509	5.88 ×	6	16.951	5.73 ×
8	12.383	7.84 ×	8	12.487	7.78 ×	8	12.341	7.87 ×
12	9.241	10.51 ×	12	8.21	11.83 ×	12	9.249	10.5 ×
16	6.224	15.6 ×	16	6.239	15.56 ×	16	6.171	15.74 ×

Table 1: Multistart MDS Performance Data

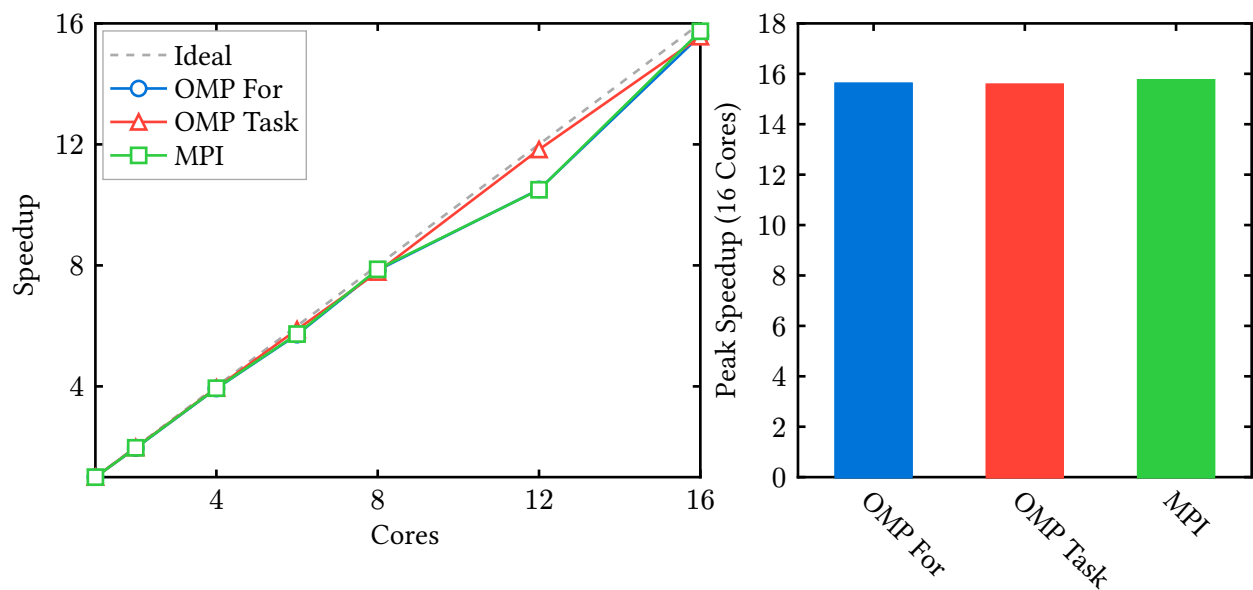


Figure 1: Multistart MDS Speedup Analysis

Παρατηρήσεις: Παρατηρούμε σχεδόν **γραμμική επιτάχυνση** (Linear Speedup) για την υλοποίηση OpenMP Loop, φτάνοντας το $15.6 \times$ στους 16 πυρήνες. Αυτό υποδεικνύει ότι ο αλγόριθμος είναι “embarrassingly parallel” και ότι η τεχνητή καθυστέρηση κυριαρχεί χρονικά, αποκρύπτοντας τα overheads δημιουργίας νημάτων. Οι υλοποιήσεις OpenMP Task και MPI παρουσιάζουν αντίστοιχη κορυφαία απόδοση στους 16 πυρήνες, επιβεβαιώνοντας ότι η επιβάρυνση επικοινωνίας στο MPI (ένα τελικό reduction) είναι αμελητέα.

2 Parallel Parametric Search in Machine Learning

2.1 Εισαγωγή & Αρχιτεκτονική

Στο δεύτερο μέρος της εργασίας υλοποιήθηκε ένα σύστημα παράλληλης αναζήτησης υπερπαραμέτρων (Random Search) για την εκπαίδευση μοντέλου λογιστικής παλινδρόμησης (Logistic Regression) με $L1$ ποινή, χρησιμοποιώντας τη βιβλιοθήκη `scikit-learn`. Ο στόχος ήταν η μελέτη διαφορετικών μοντέλων παραλληλισμού της Python για workloads που συνδυάζουν κώδικα Python και calls σε native βιβλιοθήκες (BLAS/LAPACK).

2.2 Μοντέλα Εκτέλεσης

Εξετάστηκαν οι εξής προσεγγίσεις παραλληλισμού:

1. **Threads (ThreadPoolExecutor):** Χρήση νημάτων εντός της ίδιας διεργασίας (shared memory). Παρόλο που η Python διαθέτει το Global Interpreter Lock (GIL), η `scikit-learn` (μέσω NumPy) απελευθερώνει το GIL κατά τη διάρκεια βαριών γραμμικών πράξεων, επιτρέποντας την ταυτόχρονη εκτέλεση σε πολλαπλούς πυρήνες.
2. **Processes (ProcessPoolExecutor):** Χρήση ξεχωριστών διεργασιών. Αυτή η μέθοδος παρακάμπτει πλήρως το GIL, αλλά εισάγει σημαντικό κόστος σειριοποίησης (**serialization overhead**) για τη μεταφορά δεδομένων (data copying) και αποτελεσμάτων μεταξύ των διεργασιών.
3. **MPI Futures:** Χρήση του `mpi4py.futures.MPICommExecutor`. Παρέχει high-level διεπαφή (pool of workers) αλλά κατανέμει τις εργασίες μέσω MPI σε κατανεμημένο περιβάλλον, προσφέροντας ασύγχρονη εκτέλεση.
4. **MPI Manual:** Ρητή υλοποίηση Master-Worker με χρήση `MPI_Send/MPI_Recv`. Ο Master στέλνει εργασίες στους Workers και συλλέγει αποτελέσματα, ενώ συμμετέχει και ο ίδιος στην εκτέλεση για αποφυγή αδράνειας.

2.3 Αποτελέσματα & Σχολιασμός

Η απόδοση των μοντέλων μετρήθηκε σε χρόνο και επιτάχυνση, όπως φαίνεται στον πίνακα:

Threads			Processes		
Cores	Time (s)	Speedup	Cores	Time (s)	Speedup
1	70.126	1 ×	1	72.349	0.97 ×
2	38.36	1.83 ×	2	40.023	1.75 ×
4	21.269	3.3 ×	4	23.242	3.02 ×
6	15.714	4.46 ×	6	18.153	3.86 ×
8	12.922	5.42 ×	8	16.08	4.36 ×
12	10.858	6.45 ×	12	15.828	4.43 ×
16	9.55	7.34 ×	16	15.735	4.45 ×

MPI Manual			MPI Futures		
Cores	Time (s)	Speedup	Cores	Time (s)	Speedup
1	72.752	0.96 ×	1	73.307	0.96 ×
2	44.701	1.57 ×	2	38.719	1.81 ×
4	25.243	2.78 ×	4	20.673	3.39 ×
6	23.553	2.98 ×	6	14.517	4.83 ×
8	15.19	4.61 ×	8	11.808	5.94 ×
12	13.947	5.03 ×	12	11.761	5.96 ×
16	12.644	5.54 ×	16	8.931	7.85 ×

Table 2: Random Search Performance Data

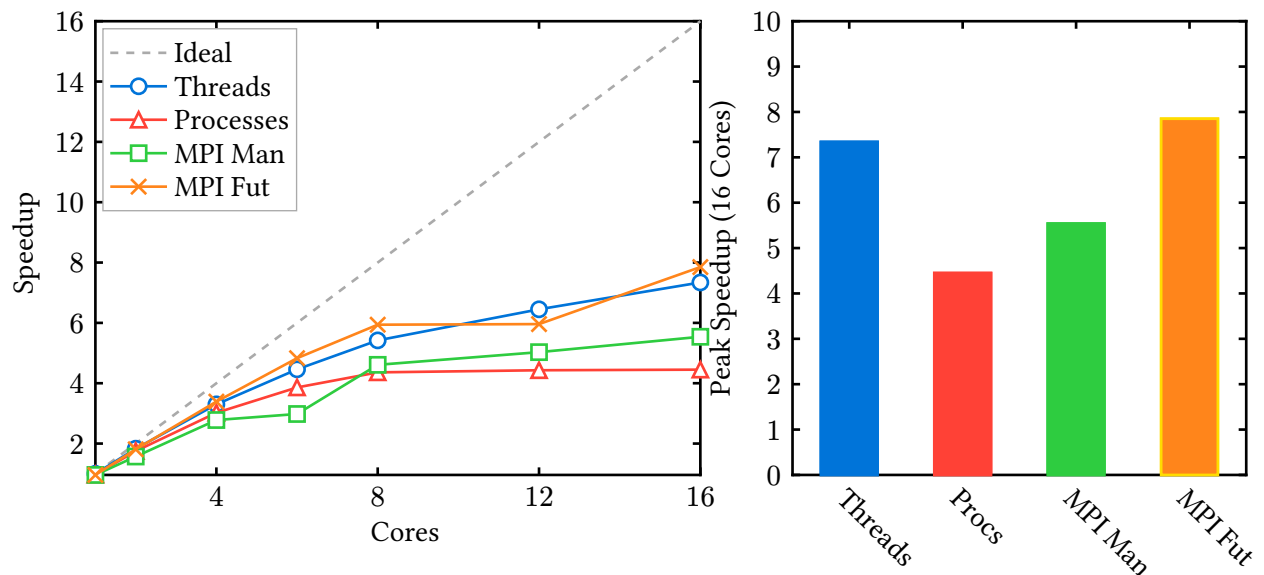


Figure 2: Random Search Speedup Analysis

Συμπεράσματα: Η προσέγγιση **MPI Futures** σημείωσε την καλύτερη επίδοση (7.85 ×), ακολουθούμενη στενά από τα **Threads** (7.34 ×).

- Η καλή επίδοση των **Threads** επιβεβαιώνει ότι το `scikit-learn` διαχειρίζεται αποδοτικά το `GIL`, επιτρέποντας τον παραλληλισμό σε επίπεδο `C/C++`.
- Αντίθετα, η προσέγγιση **Processes** είχε τη χαμηλότερη επίδοση (4.45 ×) λόγω του υψηλού κόστους (overhead) δημιουργίας διεργασιών και αντιγραφής δεδομένων.

- Η χειροκίνητη υλοποίηση **MPI Manual**, αν και λειτουργική, υστέρησε ελαφρώς έναντι της βελτιστοποιημένης `MPICommExecutor`, πιθανώς λόγω αναποτελεσματικής διαχείρισης των πόρων ή αδράνειας κατά το κλείδωμα των μηνυμάτων.